

DOCUMENTO DE APOIO AO DESENVOLVIMENTO

CampusHub

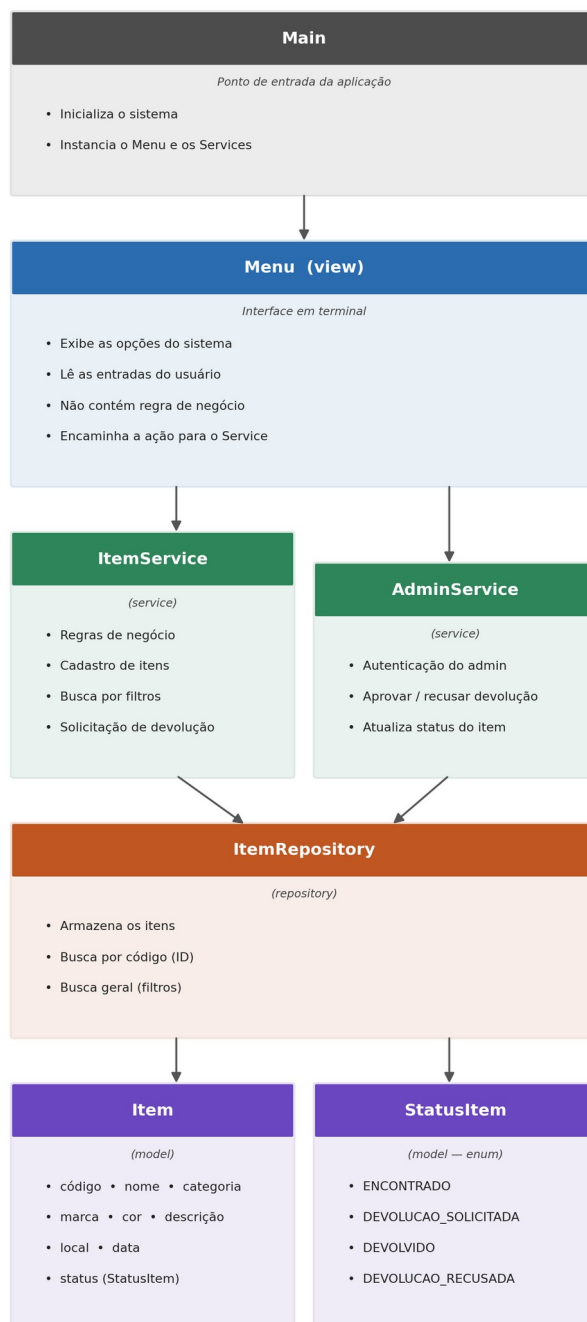
Arquitetura do Módulo de Achados e Perdidos

Estrutura em camadas • Fluxos de execução • Protótipo em Java (terminal)

1. Visão geral da arquitetura em camadas

O sistema é organizado em camadas com responsabilidades bem definidas. Essa separação segue um princípio central da Orientação a Objetos: cada classe deve ter um único motivo para mudar (responsabilidade única). Isso evita, por exemplo, que uma regra de negócio fique "escondida" dentro do menu, ou que a lógica de armazenamento se misture com a lógica de validação.

De cima para baixo, cada camada só conhece a camada imediatamente abaixo dela — o Menu não sabe como os itens são armazenados, e o ItemService não sabe como o menu exibe as opções na tela. Essa independência é o que torna o sistema fácil de evoluir: trocar a lista em memória por um banco de dados no futuro (RNF05/RNF07) vai exigir mudanças apenas no ItemRepository, sem tocar no Menu ou nos Services.



2. Responsabilidade de cada camada

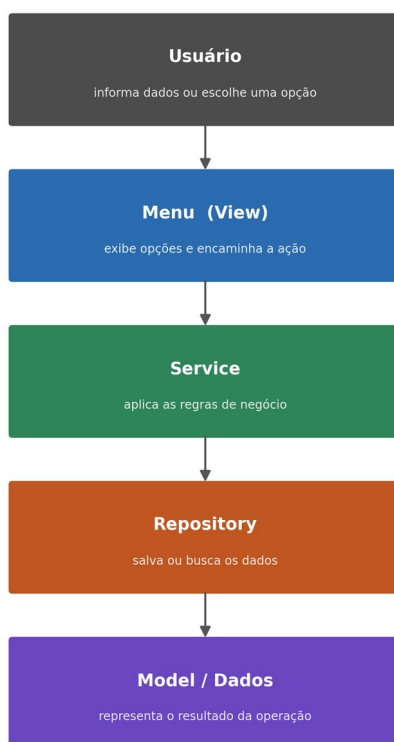
Camada	Classe(s)	Responsabilidade
Entrada	Main	Ponto de entrada do programa. Cria as instâncias necessárias (Menu, Services) e inicia a execução. Não contém lógica própria.
View	Menu	Exibe as opções em terminal, lê o que o usuário digita e chama o método correspondente no Service. Não decide regras — só recebe e repassa.
Service	ItemService, AdminService	ItemService cuida do cadastro, busca, geração de códigos, solicitações de devolução e alteração de status dos objetos. AdminService cuida exclusivamente da autenticação e do acesso administrativo. Essa divisão evita que as regras de negócio fiquem misturadas com a interface.
Repository	ItemRepository	Guarda a lista de itens e sabe como buscar neles (por código ou por filtros), além de localizar as solicitações pendentes. Isola o Service de saber se os dados estão em uma List, um arquivo ou um banco.
Model	Item, StatusItem	Item representa o objeto do mundo real (a mochila, o carregador etc.) com seus atributos. StatusItem é um enum que representa, de forma controlada, os possíveis estados desse objeto. Nenhuma das duas classes contém lógica de negócio.

Atualmente, os dados são armazenados em memória por meio de uma List dentro do ItemRepository — ou seja, todos os objetos cadastrados são perdidos quando a aplicação é encerrada. Essa é uma limitação conhecida e assumida do protótipo, e é justamente o motivo de a lógica de armazenamento ficar isolada nessa camada: substituí-la por um arquivo ou banco de dados no futuro não deve afetar o restante do sistema.

Essa divisão é chamada de arquitetura em camadas (layered architecture) e é uma versão simplificada do padrão usado em aplicações Spring Boot — o que significa que, quando você chegar no Spring Boot no seu plano de estudos, essa mesma lógica de separação (Controller/View → Service → Repository → Model) vai parecer familiar.

3. Fluxo geral de uma operação

Independentemente de qual funcionalidade o usuário está executando, toda operação do sistema segue o mesmo caminho entre as camadas: o usuário interage com o Menu, o Menu aciona o Service correspondente, o Service aplica as regras de negócio e recorre ao Repository quando precisa consultar ou alterar dados, e o Repository opera sobre o Model.

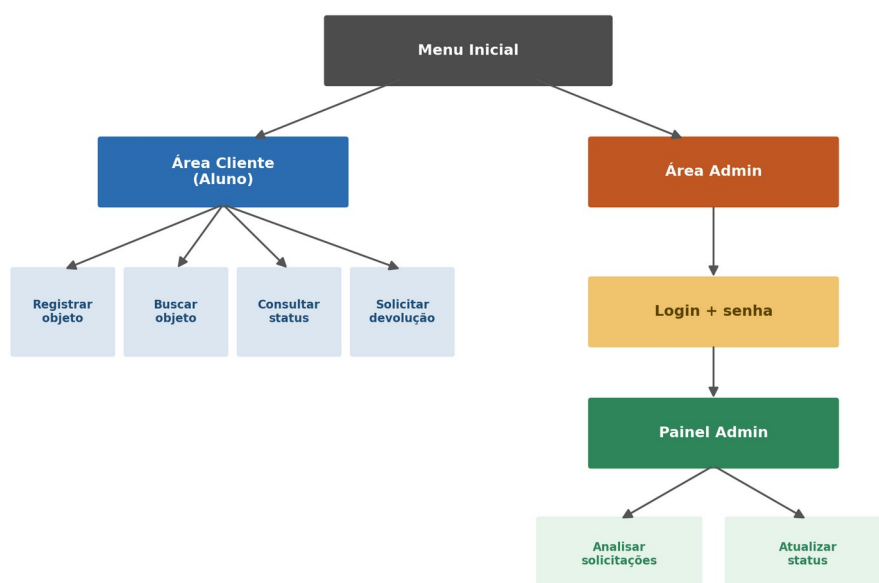


Por exemplo, ao solicitar a devolução de um objeto, o usuário informa o código através do Menu. O `ItemService` recebe essa informação, verifica as regras de negócio e utiliza o `ItemRepository` para localizar o objeto. Caso a operação seja válida, o status do objeto é atualizado — exatamente o fluxo detalhado na seção 5.2.

4. Fluxo de navegação do usuário

O menu principal direciona o usuário para duas áreas distintas: a área do aluno, com as ações do dia a dia, e a área do administrador, protegida por login, onde ficam as ações de moderação (analisar solicitações e atualizar status).

CampusHub — Fluxo de Navegação (Achados e Perdidos)

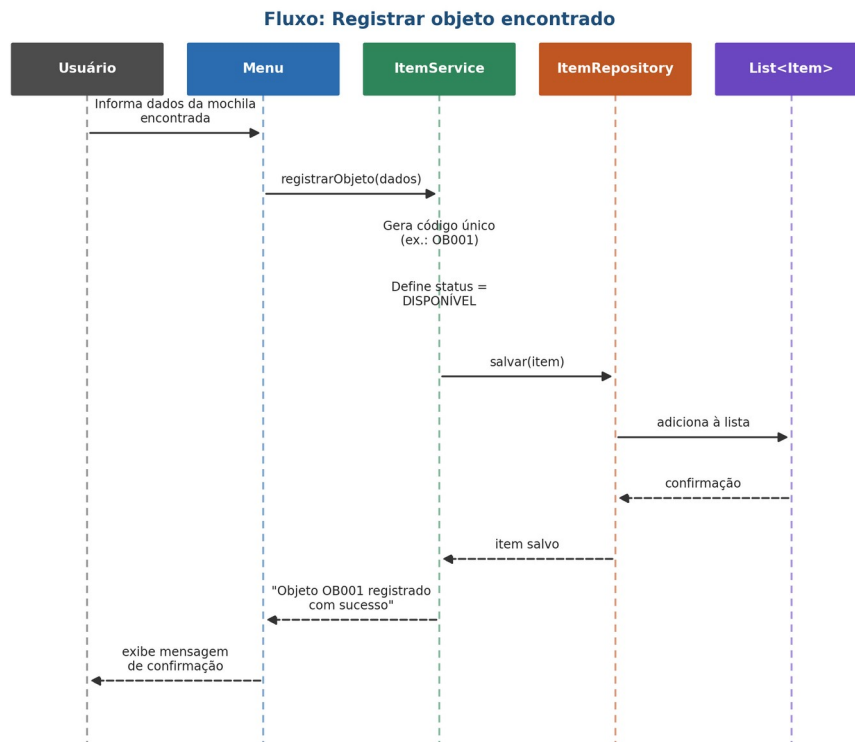


5. Fluxos de execução (passo a passo)

Os diagramas a seguir mostram, mensagem por mensagem, o que acontece internamente quando o usuário realiza duas das ações mais importantes do sistema. A seta contínua representa uma chamada de método; a seta tracejada representa o retorno dessa chamada.

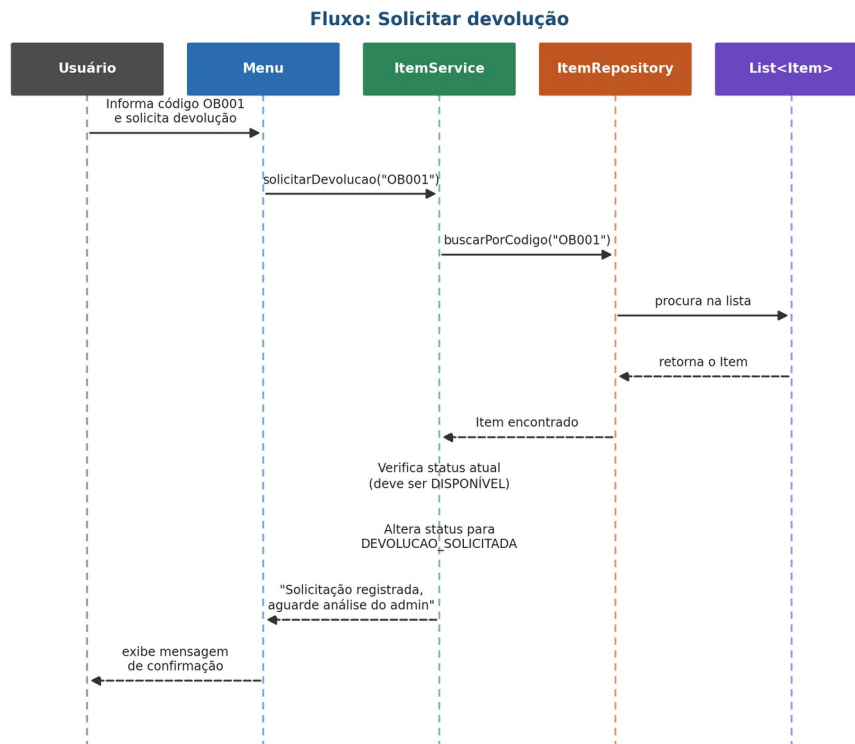
5.1 Registrar objeto encontrado

Quando o aluno informa os dados de um item encontrado, o Menu repassa esses dados para o ItemService, que é responsável por gerar o código único (RF03) e definir o status inicial como "Disponível" (RN01), antes de pedir ao ItemRepository que guarde o item na lista.



5.2 Solicitar devolução

Quando o aluno informa o código de um item e solicita a devolução, o ItemService primeiro busca o item no ItemRepository, verifica se o status atual permite a solicitação, e então altera o status para "Em análise" (RN03) — deixando o item visível ao administrador para validação posterior (RF07).



6. Por que organizar o código dessa forma?

- Baixo acoplamento: cada camada pode ser alterada isoladamente. Trocar a interface de terminal por uma interface web no futuro não exige tocar no ItemService.
- Testabilidade: como o ItemService não depende do Menu, é possível testar as regras de negócio sozinhas, sem precisar simular entradas de teclado.
- Evolução prevista no escopo: a troca da lista em memória por persistência em arquivo ou banco de dados (escopo futuro do CampusHub) fica isolada dentro do ItemRepository.
- Separação dentro do próprio Service: ItemService cuida dos objetos e devoluções, AdminService cuida apenas da autenticação — evitando que regras de negócio distintas fiquem misturadas na mesma classe.
- Leitura mais fácil: qualquer pessoa (inclusive você, daqui a alguns meses) consegue localizar rapidamente onde mexer, porque cada classe tem um propósito único e nomeado.